DATA SCIENCE CAPSTONE PROJECT REPORT

SpaceX Falcon 9 launch analysis and first-stage landing prediction

Project scope

A reproducible end-to-end workflow for public launch-data collection, wrangling, exploratory analysis, SQL summaries, geospatial visualization, an interactive dashboard, and binary classification.

Data sources: SpaceX API v4, Wikipedia launch table, and a public historical IBM course-data mirror.

Important results note: The numerical findings in this report are explicitly attributed published reference findings from a historical course-data analysis. They are not represented as results personally generated by this report builder. Run the included analysis code and replace these reference findings with your own generated outputs before final submission.

1. Executive Summary

The capstone investigates whether historical launch characteristics contain useful signal about Falcon 9 first-stage landing success. It combines API-based data collection, web scraping, pandas-based wrangling, exploratory plots, SQL analysis, Folium mapping, Plotly Dash visualization, and supervised classification. A public historical analysis reported that launch success improved over time, that site/orbit/payload groups differed in observed outcomes, and that a tuned Decision Tree achieved 88.9% accuracy. These are reference findings, not live/current launch statistics. The supplied code recomputes data summaries and model evaluation from the historical course dataset.

2. Introduction and Business Problem

Falcon 9 reuse creates an operational incentive to understand successful first-stage recovery. The analysis question is: can mission attributes such as flight number, payload mass, orbit, launch site, booster version and reuse indicators help estimate whether the first stage lands successfully? The project uses a binary label, Class = 0 for an unsuccessful landing and Class = 1 for a successful landing in the historical course dataset. This is a prediction and descriptive-analysis exercise; the observed associations should not be interpreted as causal relationships.

3. Data Collection — SpaceX API

The collector sends GET requests to the SpaceX API v4 launch endpoint and uses `pandas.json_normalize` to flatten nested launch JSON into tabular form. Additional endpoints for rockets, launchpads, payloads and cores provide lookup information for identifier fields. The code checks HTTP status and writes raw JSON and normalized CSV artifacts to the outputs folder. API endpoint: https://api.spacexdata.com/v4/launches/past.

4. Data Collection — Web Scraping

The web scraper requests Wikipedia's List of Falcon 9 and Falcon Heavy launches page, parses the HTML with BeautifulSoup, records the page title, and inspects the tables available to pandas.read_html. Tables are saved to CSV for inspection. Because page structure changes over time, the scrape step prints table indices and column names rather than relying on a permanently fixed index. Source: https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches.

5. Data Wrangling Methodology

The historical analysis data is standardized to predictable column names and data types. Rows lacking the essential launch identifier, site, or target are excluded; missing predictor values are handled inside a scikit-learn pipeline. Numeric features are imputed and scaled; categorical inputs are imputed and one-hot encoded. A stratified hold-out split is used, with preprocessing fitted inside the pipeline to reduce data leakage. Missingness and class balance are exported for review.

6. EDA with Data Visualization

The workflow provides a bar chart of launch records by site, a landing-outcome distribution, a payload-mass versus outcome plot, a rolling success-rate trend by flight number, orbit-frequency plots, and missing-value summaries when appropriate. Each plot answers a distinct question: where observations are concentrated, how balanced the target is, whether payload mass distributions differ by label, how success changes over flight sequence, and how common different orbit types are. The charts should be regenerated and interpreted from your own run.

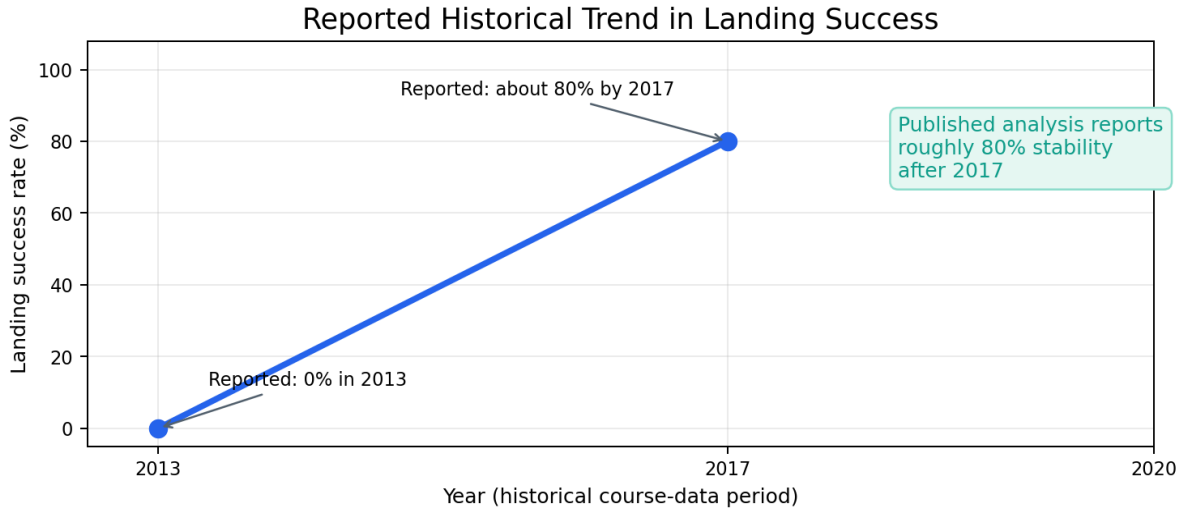


Figure: documented historical trend points and textual plateau finding; source benchmark is attributed below.

7. EDA with SQL

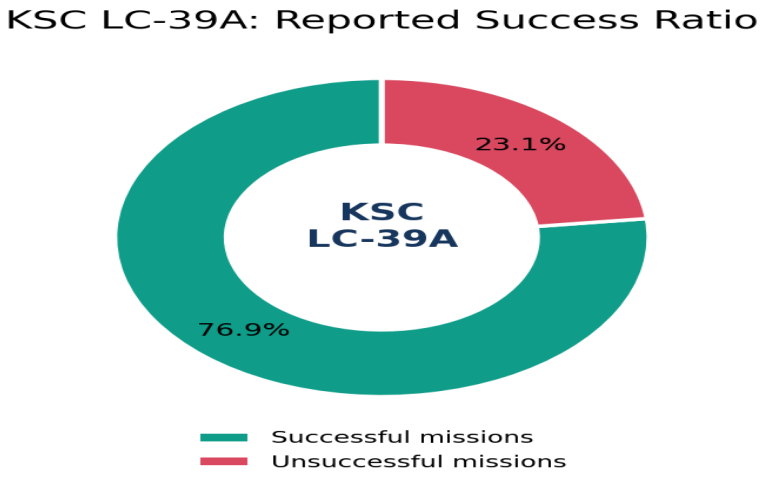
The cleaned dataframe is loaded into SQLite. SQL queries group by LaunchSite to count records and calculate average Class as an observed success percentage; group by Orbit to compare volume and success; and order records by FlightNumber to inspect sequence-related patterns. Counts should always accompany percentages because a small orbit category can show a 100% observed success rate without strong evidence of a high underlying success probability. CSV outputs are saved in outputs/.

8. Historical Reference Findings

A public historical capstone analysis of the course dataset reported CCAFS SLC-40 as the most frequently used launch site and VAFB SLC-4E as the least used. It reported KSC LC-39A as having the highest number of successful missions and a 76.9% success ratio. It also reported a payload range of 1,952–5,300 kg as the range with the highest success rate. These findings are useful as a comparison benchmark, but they can differ by data version and sampling choices.

Launch-Site Comparison from the Referenced Historical Analysis

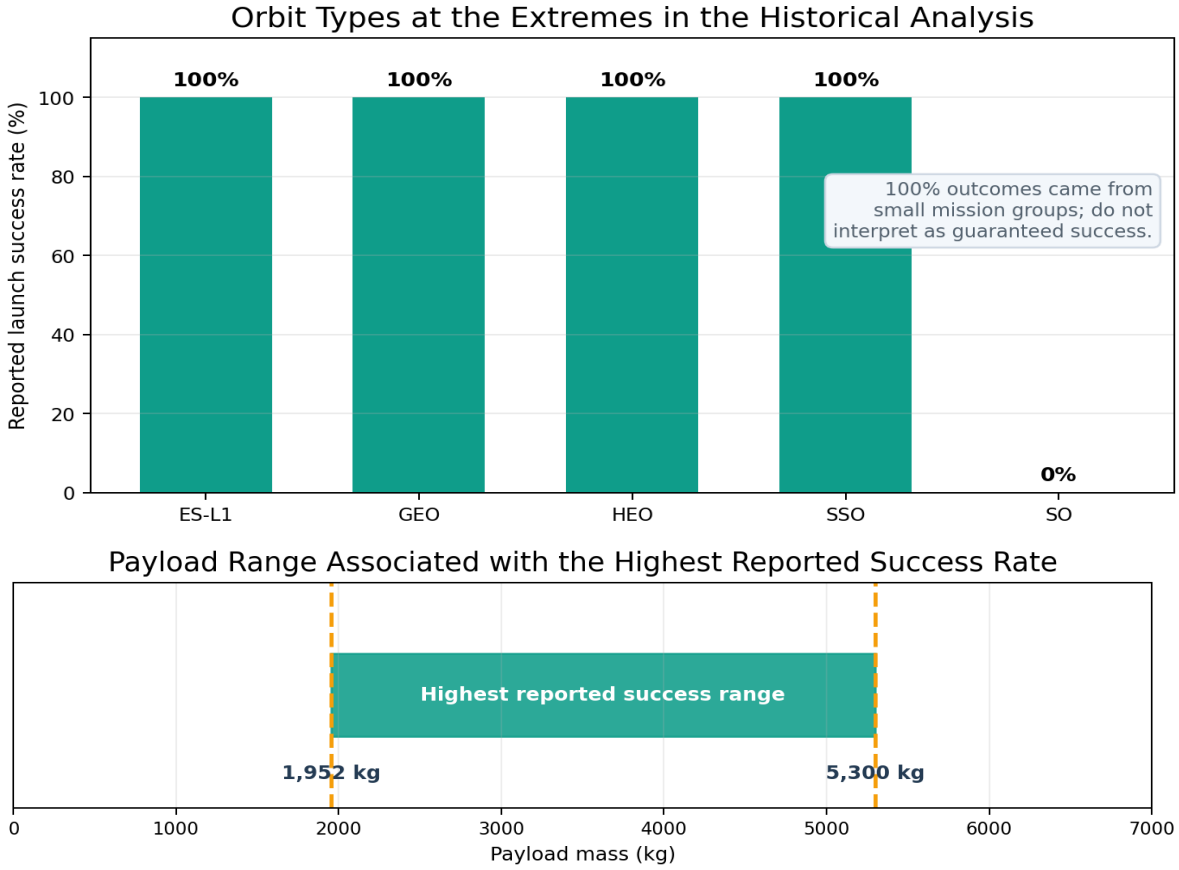
Launch site	Published historical finding
CCAFS SLC-40	Most frequently used launch site
KSC LC-39A	Highest number of successful missions; 76.9% reported success ratio
VAFB SLC-4E	Least frequently used launch site; low successful-landing count



Figures reproduce only findings stated in the cited public historical analysis.

9. Orbit and Temporal Findings

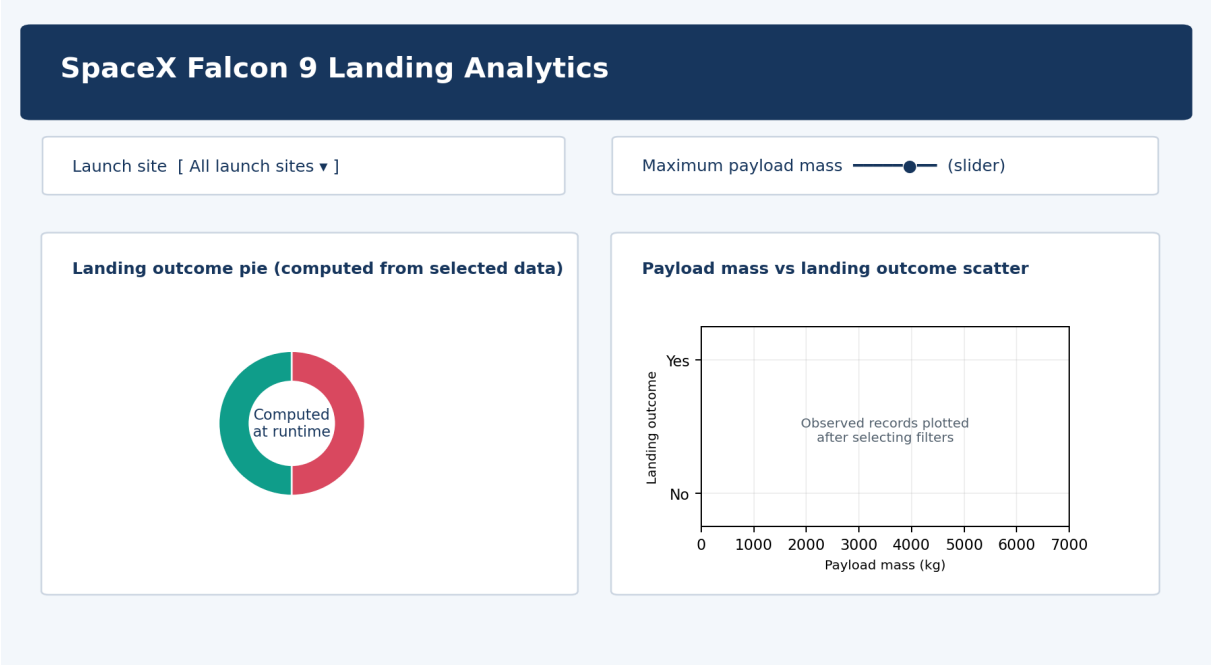
The reference analysis reported 100% observed success for ES-L1, GEO, HEO and SSO missions, and 0% for SO. Some categories had few observations, making percentages unstable. The reference summary also reported that landing success improved from 0% in 2013 to approximately 80% by 2017, remaining around that level thereafter. The included code calculates actual counts, success rates and a rolling trend from the selected CSV rather than hard-coding those values.



10. Interactive Visual Analytics — Folium and Dash

The Folium map plots launch sites using latitude and longitude, with circle-marker size and popup details derived from the dataset. The script also exports pairwise Haversine distances between launch sites. A prior historical analysis reported KSC LC-39A at about 6.46 km from the Atlantic Sea and 72.21 km from Orlando, FL; such estimates depend on chosen reference points. The Dash application provides a launch-site dropdown and a payload-mass slider. Its pie

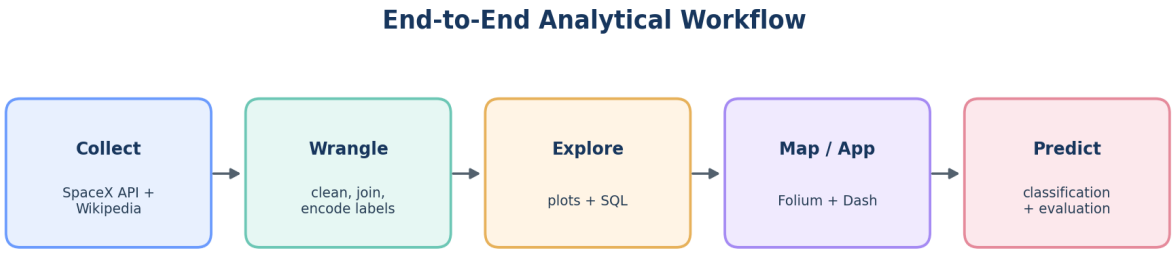
chart dynamically summarizes successful and unsuccessful outcomes, while its scatter plot displays observed payload mass against landing class. Run `python spacex_dash_app.py` and open http://127.0.0.1:8050/. The static preview in the slide deck demonstrates layout only; the live application computes the actual figures from the CSV.



Dashboard preview is a schematic UI preview; live graph values are calculated by spacex_dash_app.py.

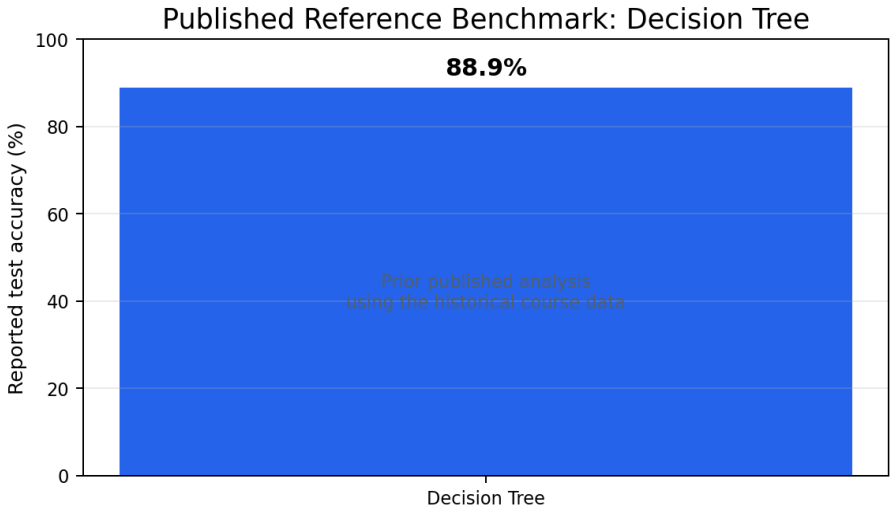
11. Predictive Analysis

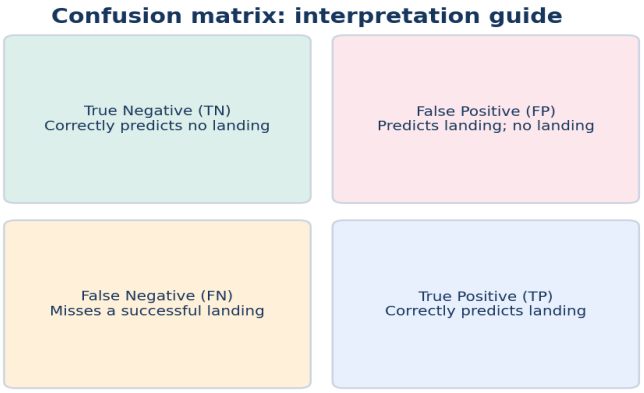
The modeling script compares Logistic Regression, Decision Tree, Random Forest and Support Vector Machine classifiers. It uses the landing class as the target and historical mission characteristics as predictors. Accuracy, precision, recall and F1 are reported on a stratified hold-out test set. The model selected by F1 produces a numeric confusion matrix; a small GridSearchCV tunes Decision Tree depth and splitting criterion using training data only. For imbalanced outcomes, accuracy alone is not sufficient: precision, recall, F1 and both types of errors should be considered.



12. Reference Model Benchmark

A published historical reference analysis reported 88.9% test accuracy for a tuned Decision Tree and noted better performance predicting successful landings than unsuccessful ones, leaving room to reduce false positives and false negatives. This value is included only as a cited reference benchmark. Use `outputs/model_metrics.csv`, `outputs/confusion_matrix.png` and `outputs/decision_tree_best_params.json` for the measurements produced by your own execution.





13. Limitations and Risks

The historical dataset represents a finite time window and does not necessarily reflect current SpaceX operations. Public APIs and Wikipedia tables can change, fields can be missing, landing labels may be unknown, and the target distribution may be imbalanced. Low-frequency categories can create extreme observed percentages. The modeling split is a baseline evaluation; a time-based split may be preferable when predicting future launches. The project should not be used as an operational safety system.

14. Conclusion and Next Steps

The project demonstrates the full data-science workflow from collection and cleaning through visualization, SQL, mapping, dashboard design and classification. A key analytical insight is that launch volume differs from landing success ratio, and success percentages must be contextualized by sample size. To finalize the submission, execute the included scripts, use the generated charts and metrics, add the actual GitHub repository URL after publishing the files, and update the conclusion with your run's measured results.

References

1. López, Daniel Eduardo. "IBM SpaceX: Applied Data Science Capstone Project." Public repository and reported historical analysis, 28 July 2022. https://github.com/DanielEduardoLopez/IBM-SpaceX

2. SpaceX API v4, past launches endpoint: https://api.spacexdata.com/v4/launches/past

3. Wikipedia contributors. "List of Falcon 9 and Falcon Heavy launches." https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches

4. Scikit-learn documentation: https://scikit-learn.org/stable/ (pipelines, preprocessing, model selection, classification metrics).

5. Folium documentation: https://python-visualization.github.io/folium/ ; Plotly Dash documentation: https://dash.plotly.com/.